



Python RegEx

[< Previous](#)[Next >](#)

A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.

RegEx can be used to check if a string contains the specified search pattern.

RegEx Module

Python has a built-in package called `re`, which can be used to work with Regular Expressions.

Import the `re` module:

```
import re
```

RegEx in Python



Search the string to see if it starts with "The" and ends with "Spain":

```
import re

txt = "The rain in Spain"
x = re.search("^The.*Spain$", txt)
```



Try it Yourself »

RegEx Functions

The **re** module offers a set of functions that allows us to search a string for a match:

Function	Description
<u>findall</u>	Returns a list containing all matches
<u>search</u>	Returns a <u>Match object</u> if there is a match anywhere in the string
<u>split</u>	Returns a list where the string has been split at each match
<u>sub</u>	Replaces one or many matches with a string



Metacharacters

Metacharacters are characters with a special meaning:

Character	Description	Example	Try it
[]	A set of characters	"[a-m]"	Try it »
\	Signals a special sequence (can also be used to escape special characters)	"\d"	Try it »
.	Any character (except newline character)	"he..o"	Try it »
^	Starts with	"^hello"	Try it »
\$	Ends with	"planet\$"	Try it »
*	Zero or more occurrences	"he.*o"	Try it »
+	One or more occurrences	"he.+o"	Try it »
?	Zero or one occurrences	"he.?o"	Try it »
{}	Exactly the specified number of occurrences	"he.{2}o"	Try it »
	Either or	"falls stays"	Try it »
()	Capture and group		

Flags

You can add flags to the pattern when using regular expressions.



re.DEBUG		Returns debug information	Try it »
re.DOTALL	re.S	Makes the . character match all characters (including newline character)	Try it »
re.IGNORECASE	re.I	Case-insensitive matching	Try it »
re.MULTILINE	re.M	Returns matches at the start/end of each line	Try it »
re.NOFLAG		Specifies that no flag is set for this pattern	
re.UNICODE	re.U	Returns Unicode matches. This is default from Python 3. For Python 2: use this flag to return only Unicode matches	Try it »
re.VERBOSE	re.X	Allows whitespaces and comments inside patterns. Makes the pattern more readable	Try it »

Special Sequences

A special sequence is a `\` followed by one of the characters in the list below, and has a special meaning:

Character	Description	Example	Try it
<code>\A</code>	Returns a match if the specified characters are at the beginning of the string	<code>"\AThe"</code>	Try it »
<code>\b</code>	Returns a match where the specified characters are at the beginning or at the end of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	<code>r"\bain"</code> <code>r"ain\b"</code>	Try it » Try it »
<code>\B</code>	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word	<code>r"\Bain"</code> <code>r"ain\B"</code>	Try it » Try it »



digits (numbers from 0-9)

\D	Returns a match where the string DOES NOT contain digits	"\D"	Try it »
\s	Returns a match where the string contains a white space character	"\s"	Try it »
\S	Returns a match where the string DOES NOT contain a white space character	"\S"	Try it »
\w	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	"\w"	Try it »
\W	Returns a match where the string DOES NOT contain any word characters	"\W"	Try it »
\Z	Returns a match if the specified characters are at the end of the string	"Spain\Z"	Try it »

Sets

A set is a set of characters inside a pair of square brackets `[]` with a special meaning:

Set	Description	Try it
[arn]	Returns a match where one of the specified characters (a , r , or n) is present	Try it »
[a-n]	Returns a match for any lower case character, alphabetically between a and n	Try it »
[^arn]	Returns a match for any character EXCEPT a , r , and n	Try it »
[0123]	Returns a match where any of the specified digits (0 , 1 , 2 , or 3) are present	Try it »



[a-zA-Z] Returns a match for any character alphabetically between **a** and **z**, lower case OR upper case [Try it »](#)

[+] In sets, **+**, *****, **.**, **|**, **()**, **\$**, **{ }** has no special meaning, so **[+]** means: return a match for any **+** character in the string [Try it »](#)

The findall() Function

The **findall()** function returns a list containing all matches.

Example

Print a list of all matches:

```
import re

txt = "The rain in Spain"
x = re.findall("ai", txt)
print(x)
```

[Try it Yourself »](#)

The list contains the matches in the order they are found.

If no matches are found, an empty list is returned:

Example

Return an empty list if no match was found:

```
import re
```

[Try it Yourself »](#)

The search() Function

The `search()` function searches the string for a match, and returns a Match object if there is a match.

If there is more than one match, only the first occurrence of the match will be returned:

Example

Search for the first white-space character in the string:

```
import re

txt = "The rain in Spain"
x = re.search("\s", txt)

print("The first white-space character is located in position:", x.start())
```

[Try it Yourself »](#)

If no matches are found, the value `None` is returned:

Example

Make a search that returns no match:

```
import re
```

[Try it Yourself »](#)

The split() Function

The `split()` function returns a list where the string has been split at each match:

Example

Split at each white-space character:

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

[Try it Yourself »](#)

You can control the number of occurrences by specifying the `maxsplit` parameter:

Example

Split the string only at the first occurrence:

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt, 1)
print(x)
```




The sub() Function

The `sub()` function replaces the matches with the text of your choice:

Example

Replace every white-space character with the number 9:

```
import re

txt = "The rain in Spain"
x = re.sub("\s", "9", txt)
print(x)
```

[Try it Yourself »](#)

You can control the number of replacements by specifying the `count` parameter:

Example

Replace the first 2 occurrences:

```
import re

txt = "The rain in Spain"
x = re.sub("\s", "9", txt, 2)
print(x)
```

[Try it Yourself »](#)



Note: If there is no match, the value None will be returned, instead of the Match Object.

Example

Do a search that will return a Match Object:

```
import re

txt = "The rain in Spain"
x = re.search("ai", txt)
print(x) #this will print an object
```

Try it Yourself »

The Match object has properties and methods used to retrieve information about the search, and the result:

.span() returns a tuple containing the start-, and end positions of the match.

.string returns the string passed into the function

.group() returns the part of the string where there was a match

Example

Print the position (start- and end-position) of the first match occurrence.

The regular expression looks for any words that starts with an upper case "S":

```
import re

txt = "The rain in Spain"
```

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Get Certified](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[W3.CSS](#)[C](#)[C++](#)[C#](#)[Try it Yourself »](#)

Example

Print the string passed into the function:

```
import re

txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.string)
```



Example

Print the part of the string where there was a match.

The regular expression looks for any words that starts with an upper case "S":

```
import re

txt = "The rain in Spain"
x = re.search(r"\bS\w+", txt)
print(x.group())
```

[Try it Yourself »](#)

Note: If there is no match, the value None will be returned, instead of the Match Object.

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Get Certified](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[W3.CSS](#)[C](#)[C++](#)[C#](#)

Consider the following code:

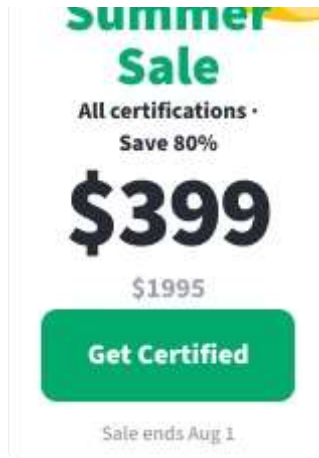
```
import re
txt = 'The rain in Spain'
x = re.findall('[a-c]', txt)
print(x)
```

What will be the printed result?

- ☐ ['a', 'a']
- ☐ 'The rin in Spin'
- ☐ 2

[Submit Answer »](#)

[◀ Previous](#)[Sign in to track progress](#)[Next >](#)

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Get Certified](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[W3.CSS](#)[C](#)[C++](#)[C#](#)[PLUS](#)[SPACES](#)[GET CERTIFIED](#)[FOR TEACHERS](#)[BOOTCAMPS](#)[CONTACT US](#)

Top Tutorials

- [HTML Tutorial](#)
- [CSS Tutorial](#)
- [JavaScript Tutorial](#)
- [How To Tutorial](#)
- [SQL Tutorial](#)
- [Python Tutorial](#)
- [W3.CSS Tutorial](#)
- [Bootstrap Tutorial](#)
- [PHP Tutorial](#)
- [Java Tutorial](#)

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Get Certified](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[W3.CSS](#)[C](#)[C++](#)[C#](#)

[CSS Reference](#)
[JavaScript Reference](#)
[SQL Reference](#)
[Python Reference](#)
[W3.CSS Reference](#)
[Bootstrap Reference](#)
[PHP Reference](#)
[HTML Colors](#)
[Java Reference](#)
[AngularJS Reference](#)
[jQuery Reference](#)

Top Examples

[HTML Examples](#)
[CSS Examples](#)
[JavaScript Examples](#)
[How To Examples](#)
[SQL Examples](#)
[Python Examples](#)
[W3.CSS Examples](#)
[Bootstrap Examples](#)
[PHP Examples](#)
[Java Examples](#)
[XML Examples](#)
[jQuery Examples](#)

Get Certified

[HTML Certificate](#)
[CSS Certificate](#)
[JavaScript Certificate](#)
[Front End Certificate](#)
[SQL Certificate](#)
[Python Certificate](#)
[PHP Certificate](#)
[jQuery Certificate](#)
[Java Certificate](#)
[C++ Certificate](#)
[C# Certificate](#)
[XML Certificate](#)

[FORUM](#) [ABOUT](#) [ACADEMY](#)

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.

Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using W3Schools, you agree to have read and accepted our [terms of use](#), [cookies](#) and [privacy policy](#).

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.